Парсер вакансий Habr Career: требования и архитектура

Oct 14, 2026 · @Eiden

Цель и границы

Проект каждый день собирает вакансии Habr Career по выбранным направлениям, хранит их вместе с историей и по накопленным данным отвечает на три вопроса: **какие навыки чаще всего просят у junior, сколько за это платят и как меняется спрос по неделям**. Это не выгрузка в таблицу для ручной фильтрации: ценность в данных за несколько недель и в аккуратных выводах, которых сайт сам не показывает.

Проект сознательно небольшой: его можно закончить за 6–8 недель, и он остаётся понятным целиком.

Делаем:

- ежедневный сбор вакансий выбранных категорий;
- хранение в базе с историей изменений;
- простую очистку данных: зарплата, грейд, формат работы, навыки;
- 4–6 графиков и короткий разбор с честно описанными ограничениями.

Не делаем: другие площадки, Telegram-бота, ML-модели, сложную инфраструктуру, сбор данных о людях. Эти идеи можно вернуть позже, когда основа заработает.

Критерий успеха: проект запускается по README одной-двумя командами, сбор идёт автоматически не меньше 6–8 недель, по данным построены графики и написан разбор.

Что происходит с данными после парсера

Парсер возвращает не готовые данные, а **разобранную запись**: поля так, как они написаны на сайте, обычно текстом. Дальше запись проходит несколько простых шагов, и на каждом у неё свой формат. Так ошибку в одном шаге можно исправить, не ломая остальные.

Шаг	Получает	Отдаёт	Правила
1. Сбор	URL	Сырая страница в файле	Не менять и не чистить; сохранять всегда
2. Парсинг	Сырой файл	Разобранная запись: поля как на сайте	Только извлечение, ничего не угадывать; нет поля — `NULL`
3. Очистка	Разобранная запись	Чистая запись	Зарплата в числа, грейд и формат работы в единый вид, навыки в нижний регистр
4. Загрузка	Чистые записи	Строки в базе и дневной снимок	Повторный запуск за день не создаёт дублей
5. Проверка	Данные запуска	Короткая сводка	Сколько вакансий, сколько пустых полей, были ли ошибки

Ниже выдуманный пример формы записи. Реальные поля сайта проверь на этапе разведки и подгони пример под них.

Разобранная запись, выход парсера:

```
{
  "vacancy_id": "1000000000",
  "title": "Junior аналитик данных",
  "company": "Пример Компания",
  "salary_raw": "от 80 000 ₽",
  "location_raw": "Москва, можно удалённо",
  "grade_raw": "Junior",
  "skills_raw": ["SQL", "Python", "Postgres"],
  "url": "https://example.invalid/vacancies/1000000000"
}
```

Чистая запись, выход очистки:

```
{
  "vacancy_id": "1000000000",
  "title": "Junior аналитик данных",
  "company": "Пример Компания",
  "salary_from": 80000,
  "salary_to": null,
  "grade": "junior",
  "remote": true,
  "city": "Москва",
  "skills": ["sql", "python", "postgres"],
  "url": "https://example.invalid/vacancies/1000000000"
}
```

Что должно выполняться всегда:

- Сырые файлы не меняются и не удаляются: из них можно заново разобрать данные после правки парсера.
- Если значение нельзя определить, оно остаётся `NULL`. Нуль или «среднее по рынку» вместо пропуска подставлять нельзя.
- Исходные строки (`salary_raw`, `grade_raw`) хранятся в базе рядом с чистыми значениями.
- Навыки приводятся только к нижнему регистру. Словарь синонимов нужен позже и только если в данных видны явные дубли (см. P3.3).

Последовательность работы

Диаграмма показывает один ежедневный запуск. Главное в ней: сборщик не передаёт страницы парсеру напрямую, а кладёт их в файлы, и вся дальнейшая работа идёт с файлами, а не с сайтом.

Обработка читает сохранённые файлы, а не ответ сайта

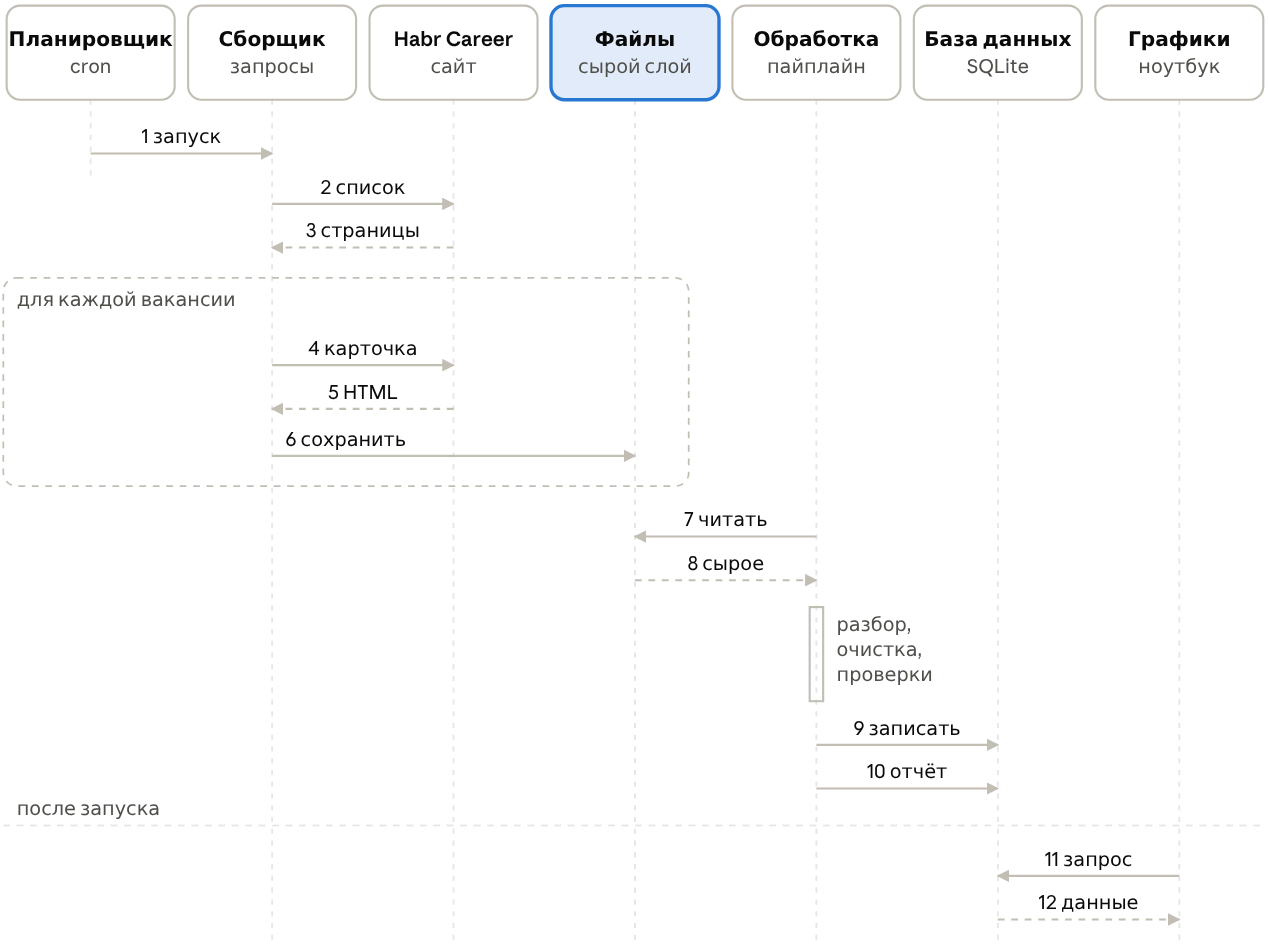


диаграмма последовательности · один ежедневный запуск, 12 шагов

Сплошная стрелка — запрос или команда, пунктирная — ответ. Номера совпадают с таблицей ниже.

№	От → к	Что происходит	Что передаётся
1	Планировщик → Сборщик	Раз в сутки стартует ежедневный запуск	Команда запуска и дата
2	Сборщик → Habr Career	Запрос страницы списка, между запросами пауза	URL страницы списка
3	Habr Career → Сборщик	Сайт отдаёт страницу списка; из неё берутся адреса карточек	HTML списка

№	От → к	Что происходит	Что передаётся
4	Сборщик → Habr Career	Для каждой вакансии: запрос карточки	URL карточки
5	Habr Career → Сборщик	Сайт отдаёт карточку	HTML карточки
6	Сборщик → Файлы	Сырой ответ сохраняется под датой запуска, ничего не меняя	Файл и метаданные: адрес, время, статус, hash
7	Обработка → Файлы	После сбора обработка берёт новые файлы за день	Список файлов
8	Файлы → Обработка	Файлы отдают сырые страницы	Сырой HTML
—	Внутри Обработки	Парсинг, очистка, проверки (раздел выше)	Разобранные и чистые записи
9	Обработка → База данных	Загрузка вакансий, дневных снимков и навыков; пометка закрытых вакансий	Чистые записи
10	Обработка → База данных	Запись отчёта запуска и результатов проверок	Сводка запуска
11	Графики → База данных	Чтение данных для графиков	SQL-запрос
12	База данных → Графики	Ответ с данными	Таблицы с данными

Ошибка на одной вакансии (шаги 4–8) пишется в лог и не останавливает запуск. Сбой на шаге 2, когда список недоступен, завершает запуск с ошибкой и уведомлением.

Функциональные требования

Требования разбиты на пять уровней, P0–P4. Каждый уровень заканчивается чем-то, что работает и что можно показать. Не начинай следующий, пока не закрыт предыдущий. Требование считается выполненным, когда проходит проверка в правом столбце, а не когда код написан.

P0. Первый запрос (1–3 дня)

Результат: скрипт скачивает одну страницу списка и одну карточку, сохраняет файлы и печатает несколько полей.

ID	Требование	Как проверить
P0.1	Сохранить страницу списка вакансий в файл	Файл открывается в браузере
P0.2	Взять из списка адрес одной вакансии, скачать и сохранить карточку	В файле текст вакансии
P0.3	Из сохранённой карточки (не из сети) извлечь название, компанию, зарплату текстом и URL	Значения совпадают с сайтом при сравнении глазами

P1. Выгрузка (около недели)

Результат: одна команда выгружает все вакансии выбранной категории в таблицу CSV.

ID	Требование	Как проверить
P1.1	Обход всех страниц списка до последней	Число вакансий совпадает со счётчиком на сайте
P1.2	Пауза между запросами и ограниченное число повторов при ошибке	В логе видны паузы; без сети скрипт завершается понятной ошибкой, а не зависает
P1.3	Сырые страницы сохраняются в папки по дате запуска	В `data/raw/ДАТА/` лежит файл на каждую вакансию

ID	Требование	Как проверить
P1.4	Парсер читает только сохранённые файлы: название, компания, зарплата текстом, город и формат работы, грейд, навыки, дата публикации, описание, URL	Для пяти вакансий поля совпадают с сайтом
P1.5	Результат записывается в CSV; повторный запуск не создаёт дублей (вакансия определяется по идентификатору из URL)	Запусти дважды подряд: число строк не выросло

P2. База и история (1–2 недели)

Результат: данные лежат в SQLite, у каждой вакансии видна история.

ID	Требование	Как проверить
P2.1	Схема базы описана в файле `schema.sql` и создаётся одной командой	На пустом файле базы команда создаёт таблицы
P2.2	Вакансия добавляется или обновляется по идентификатору	Повторная загрузка тех же данных не меняет число строк
P2.3	Каждый запуск пишет дневной снимок вакансии: дата и hash содержимого	За два дня два снимка; если изменить зарплату в тестовых данных, hash меняется
P2.4	Вакансия, которой не было в выдаче 3 дня подряд, помечается закрытой с датой	Убери вакансию из тестовой выдачи: через 3 запуска она закрыта
P2.5	Ошибка на одной вакансии не останавливает запуск и пишется в лог с адресом	Подставь повреждённый файл: запуск завершился, ошибка есть в логе

P3. Чистые данные и автозапуск (1–2 недели)

Результат: данные пригодны для расчётов, сбор идёт сам.

ID	Требование	Как проверить
P3.1	Зарплата: исходная строка сохраняется, числа «от» и «до» извлекаются, валюта хранится отдельно, нет зарплаты — `NULL`	Тесты на примерах «от X», «до Y», «X–Y» и «без зарплаты»
P3.2	Грейд и формат работы приводятся к небольшому списку значений	Нераспознанное остаётся `NULL`, их число видно в сводке
P3.3	Навыки приводятся к нижнему регистру. Если среди 50 самых частых есть явные дубли вроде «postgres» и «postgresql», объедини их маленьким словарём в одном файле	Топ-50 просмотрен вручную, явных дублей не осталось
P3.4	Перепарсинг: команда заново разбирает сохранённые файлы	После правки парсера данные обновились без запросов к сайту
P3.5	После каждого запуска пишется сводка: страницы, вакансии, новые, закрытые, ошибки; предупреждение, если вакансий стало заметно меньше	Намеренно сломай парсер: сводка показывает проблему
P3.6	Ежедневный автозапуск по расписанию	Семь дней подряд без твоего участия
P3.7	Тесты парсера на сохранённых HTML-примерах, без сети	`pytest` проходит без интернета

P4. Графики и выводы (1–2 недели)

Результат: на вопросы о рынке можно ответить цифрами и графиками.

ID	Требование	Как проверить
P4.1	Ноутбук с 4–6 графиками: частота навыков в junior-вакансиях, зарплаты по грейдам, доля удалённой работы, число вакансий по неделям, срок жизни вакансий	Ноутбук запускается от начала до конца и воспроизводит графики

ID	Требование	Как проверить
P4.2	Рядом с каждой цифрой указан размер выборки (n); зарплаты считаются только по вакансиям, где они указаны	На графиках видно n; в тексте сказано, сколько вакансий без зарплаты
P4.3	Короткий разбор (1–2 страницы) с разделом «ограничения данных»	Опубликован в README или в статье

Технологии

Начинай с простейшего варианта в каждой строке. Усложнять стоит только тогда, когда упрёшься в ограничение.

Что	Чем	Почему
Сбор	`httpx` и `selectolax` (или BeautifulSoup)	Просто и легко отлаживать. Браузер (Playwright) нужен, только если страница не отдаёт данные без него
Сырые страницы	Файлы в `data/raw/ДАТА/`	Из них можно заново разобрать данные
База	SQLite	Один файл, ничего не нужно ставить, объёма хватит с запасом. PostgreSQL можно добавить позже, если захочешь
Обработка	Python, `pandas`, SQL	Основной рабочий набор аналитика
Графики	Jupyter-ноутбук с `matplotlib` или `plotly`	Проще всего сделать и показать; Streamlit можно добавить позже
Расписание	`cron` или GitHub Actions	Простое расписание без отдельных сервисов
Качество кода	`pytest`, `ruff`, Git	Минимум, который ожидают от junior

Если постоянно включённого компьютера или сервера нет, сбор можно запускать по расписанию через GitHub Actions, но тогда файл базы и сырые страницы нужно сохранять между запусками; это решается на этапе P3.6.

Модель данных

Вакансия хранится как сама запись плюс дневные снимки. Так видно, когда она появилась, как менялась и когда закрылась. Это стартовый набор из четырёх таблиц, дополняй по необходимости.

Таблица	Основные поля	Зачем
`vacancies`	vacancy_id, title, company, grade, remote, city, salary_from, salary_to, currency, description, url, first_seen, last_seen, closed_at; исходные строки salary_raw, location_raw, grade_raw	Текущее состояние вакансии
`vacancy_skills`	vacancy_id, skill	Навыки вакансии, одна строка на навык
`vacancy_snapshots`	vacancy_id, snapshot_date, content_hash, salary_from, salary_to	Дневной срез; по смене hash видно, что изменилось
`runs`	run_id, started_at, pages, new, closed, errors	Сводка каждого запуска

Правила:

- Вакансия считается закрытой, если её не было в выдаче 3 дня подряд, а не после первого пропуска.
- Пустое значение остаётся `NULL`, а не нулём или средним.
- Зарплаты считай только по вакансиям в рублях, остальные учитывай отдельно.

Структура репозитория и правила

Начни с одного файла-скрипта и дели на модули, когда он вырастет. Структура ниже — цель, а не стартовое условие.

```
habr-vacancies/
├── README.md
├── schema.sql
├── pyproject.toml
├── src/
│   └── vacancies/
│       ├── collect.py     # запросы, пауза, сохранение сырого
│       ├── parse.py       # сырое -> разобранная запись
│       ├── clean.py       # очистка и приведение значений
│       ├── db.py          # работа с SQLite
│       └── pipeline.py    # ежедневный запуск
├── tests/
│   └── fixtures/          # сохранённые HTML-примеры
├── notebooks/             # графики и анализ
├── data/                  # сырые файлы и база (не в git)
└── docs/                  # заметки: что узнал, какие решения принял
```

Правила разработки:

- Парсинг и очистка — функции, которые получают данные и возвращают данные, без сети и базы внутри. Так их легко проверять тестами.
- Уважай сайт: прочитай `robots.txt` и условия использования (я их не проверял), держи паузу 2–3 секунды между запросами, один поток, честный User-Agent. Не публикуй сырые данные целиком, только графики и выводы.
- Один сбой не роняет весь запуск: ошибка пишется в лог, остальное продолжается.
- Настройки (категории, пауза, пути) лежат в конфиге или `.env`, а не в коде.
- Небольшие коммиты с понятными сообщениями, ветка `main` всегда рабочая.
- README: цель проекта, как запустить, пример графика, ограничения данных.

Порядок работы

Сроки в заголовках уровней — ориентир для новичка. P0–P3 вместе занимают около 5–8 недель, P4 добавляет ещё 1–2. Если идёшь медленнее, это нормально.

Запусти автосбор как можно раньше. Как только закрыто P3.6, начинай копить историю: для осмысленных выводов нужно минимум 6–8 недель данных.

Ритм на каждое требование: прочитай проверку, сделай самым простым способом, проверь, сделай небольшой коммит и запиши в `docs/`, что оказалось неожиданным. Не получается два дня подряд — раздели требование на две части и сначала сделай лёгкую.

Риски и готовность проекта

Риск	Что делать
Сайт поменяет вёрстку	Сырые страницы сохраняются всегда, тесты на примерах, сводка покажет резкое падение числа вакансий; после правки парсера перепарси старые файлы
Сайт ограничит запросы	Держи медленный темп; при блокировке не обходи защиту, а замедлись или остановись
Правила сайта запрещают сбор	Проверь на этапе разведки; при запрете выбери другой источник
Проект растёт вширь	Не переходи на следующий уровень, пока не закрыт предыдущий; новые идеи записывай в список «потом»
Пропадёт мотивация	Один маленький законченный шаг в день, публичный репозиторий с регулярными коммитами

Проект готов, когда:

- [ ] Репозиторий публичный, README объясняет цель, запуск и ограничения
- [ ] Проект запускается по README на чистой машине
- [ ] Сбор идёт автоматически не меньше 6–8 недель, пропуски задокументированы
- [ ] В базе есть история вакансий и дневные снимки
- [ ] Тесты парсера проходят без интернета
- [ ] Ноутбук воспроизводит графики, на них указан размер выборки

- [ ] Написан короткий разбор с разделом «ограничения данных»
- [ ] Ты можешь за пять минут объяснить любое своё решение